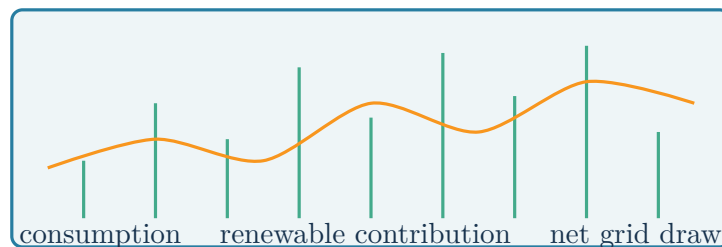


DATA ENGINEERING MINI-PROJECT

Smart Grid Pulse

A Kappa Pipeline for Live Grid Monitoring and Daily Billing



Module: EC8203 - Big Data Analytics
Member 1: Madakaladeniya I.U / EG/2021/4651
Member 2: Wijesinghe S.A / EG/2021/4877
Submission date: 30 September 2026

One simulated day equals five minutes.

Contents

1	Executive Summary	2
2	Use Case and Requirements	2
3	Architecture Decision: Kappa over Lambda	3
3.1	Decision drivers	3
3.2	Selected Kappa architecture	3
3.3	Rejected Lambda alternative	3
4	Technology Stack and Rationale	4
5	Data Ingestion and Processing	4
6	Storage and Serving Design	5
7	Observability and Failure Handling	5
8	Results and Validation	5
9	Security, Governance and Data Quality	6
10	Limitations and Production Evolution	6
11	Team Contributions and Delivery Plan	7
12	Conclusion	7
A	Reproduction and Demo Checklist	7
B	Core Data Contracts	7

1 Executive Summary

Smart Grid Pulse is an end-to-end data platform answering two operational questions: what is the current grid load and renewable contribution in each zone, and what will each household owe after daily tariff data is applied? A Python producer generates meter readings every three seconds. Kafka provides a durable replayable log; Spark Structured Streaming cleans, deduplicates, derives net grid draw and produces one-minute zone metrics. A second Python source creates a daily tariff CSV. Apache Airflow runs this source every five minutes, publishes the records as reference events, upserts the current tariffs, and creates a daily household report. PostgreSQL supports the serving workload and FastAPI exposes live and daily results. JSON logs, Prometheus metrics, alert rules, Spark checkpoints and Airflow task history make system behaviour visible.

The design uses a Kappa architecture because the continuous event log is the authoritative history and both source types can be represented as immutable events. This eliminates two competing implementations of business rules. The submitted laptop deployment deliberately uses one broker and one Spark process for clarity and reproducibility; the report identifies the controls needed at production scale.

2 Use Case and Requirements

The selected scenario is Smart Grid Energy Monitoring and Billing. Twenty simulated households belong to four grid zones. Each meter emits consumption and solar generation values plus identity, zone and event time. Once per simulated day, the tariff provider drops a file describing rate, tier and subsidy status for every household.

The live output must report total consumption, solar generation, net imported energy, renewable percentage and active households for every zone. The daily output must join accumulated readings to the latest tariff and estimate each household's bill. The key non-functional requirements are replay after failure, duplicate protection, bounded handling of late events, transparent health signals, repeatable local setup and clear operator evidence.

Table 1: Requirement interpretation and acceptance evidence

Requirement	Implementation			Evidence
Streaming ingestion	Idempotent household-keyed	Kafka events	producer; every 3s	Producer JSON logs and topic offsets
Daily source	Airflow creates one dated CSV each 5-minute simulated day			DAG run, CSV and task logs
Meaningful processing	Validation, deduplication, net-energy derivation and event-time windows			Spark job and PostgreSQL rows
Queryable output	Indexed	PostgreSQL tables	and FastAPI endpoints	Live and billing JSON responses
Observability	Structured logs, counters, latency histogram and two alert rules			Prometheus targets/rules and service logs
Reproducibility	Versioned images, Compose, schema initialization and tests			README commands and test results

3 Architecture Decision: Kappa over Lambda

3.1 Decision drivers

The dominant workload is a continuous append-only meter stream. Operators require seconds-to-minute freshness, deterministic recovery and a simple codebase that two students can explain. Daily tariff data changes reference state rather than requiring a separate historical computation engine. Kafka retention plus Spark checkpoints already provide replay, while PostgreSQL serves results without becoming the raw source of truth.

3.2 Selected Kappa architecture

All changes are events. Meter readings enter `meter-readings`; tariff rows enter `tariff-updates`, keyed by household. Spark processes the live stream and can recompute it from earlier offsets when logic changes. Airflow remains valuable: it owns the daily file boundary, publication, reference-table upsert and scheduled report. Kappa therefore describes the processing model, not the absence of orchestration or scheduled inputs.

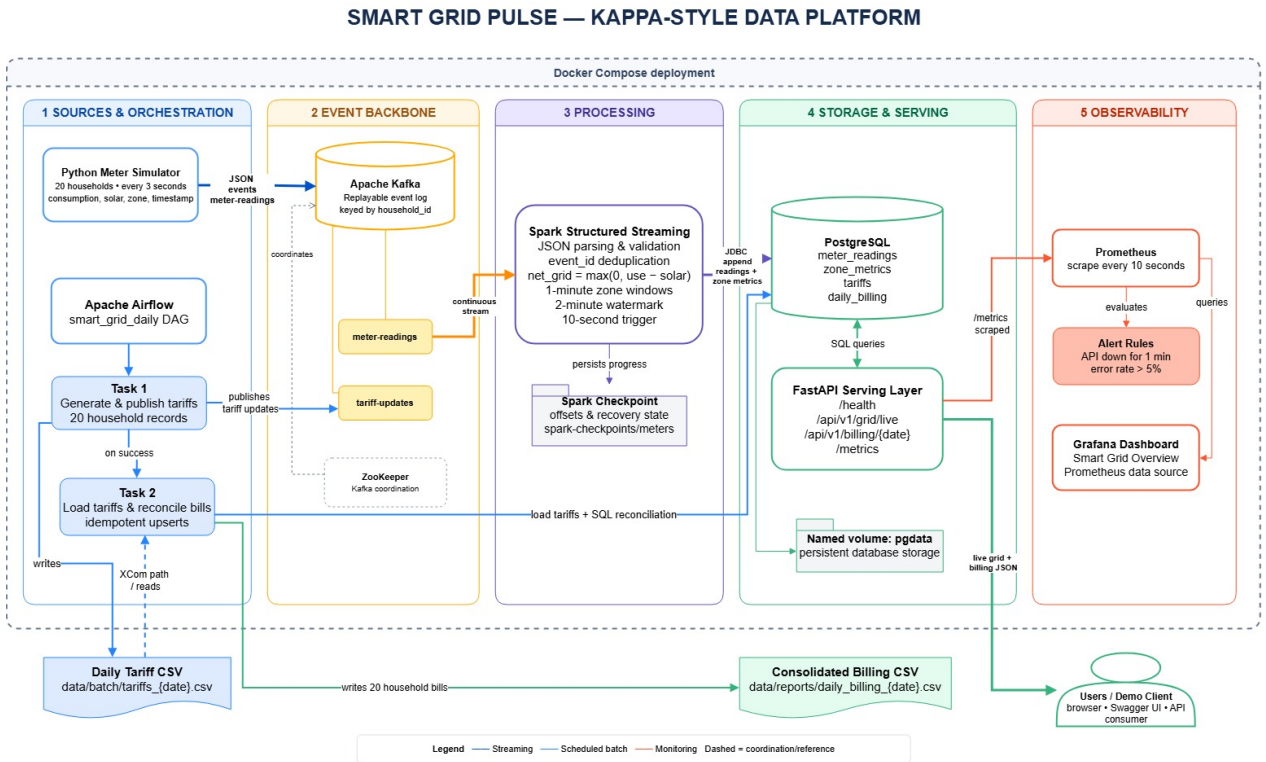


Figure 1: Implemented Smart Grid Pulse Kappa-style platform: sources and orchestration, Kafka event backbone, Spark processing, storage and serving, and observability.

3.3 Rejected Lambda alternative

A Lambda design would preserve an immutable raw layer, run a historical batch path, and merge its views with a low-latency speed path. It can offer accurate recomputation and isolate late correction, but here it creates duplicate aggregation and joining logic, two consistency models and a serving-layer merge. Those costs are disproportionate for twenty simulated meters and one small daily reference file. Kappa has a single transformation path and replay mechanism. Its trade-off is that long replays may take time and schema evolution must remain compatible with retained events.

Production mitigations include long Kafka retention or object-store archival, versioned schemas, shadow reprocessing topics and atomic view promotion.

Table 2: Architecture decision matrix (5 is best)

Driver	Weight	Kappa	Lambda	Reason
Low latency	25%	5	5	Both support a speed path
Logic simplicity	25%	5	2	One versus two processing implementations
Replay/correction	20%	4	5	Kafka replay is sufficient at this scale
Cost/operations	15%	5	2	Fewer engines and views
Serving consistency	15%	4	3	No batch/speed merge race
Weighted score		4.65	3.55	Kappa selected

4 Technology Stack and Rationale

Kafka decouples producers from processing, partitions by household for ordered per-household history, and retains offsets for replay. A production topic would use at least three partitions and replication factor three; the Compose version uses a single broker to fit a student laptop. Idempotent producer mode reduces retry duplicates, while UUID event identifiers allow downstream deduplication.

Spark Structured Streaming was chosen over Storm because its DataFrame API expresses event-time windows, watermarks and PostgreSQL micro-batch writes concisely. Checkpoints persist offsets and progress. The two-minute watermark bounds state for delayed events. The `foreachBatch` sink makes SQL writes explicit, though production would stage each batch and merge transactionally for stronger exactly-once semantics.

Airflow models the daily boundary and exposes runs, dependencies, duration and failure. Its five-minute schedule compresses one day for demonstration. PostgreSQL provides constraints, indexes, joins and familiar query serving. FastAPI supplies typed discoverable endpoints and a Prometheus exposition endpoint. Prometheus evaluates availability and error-rate rules; Grafana provides the visualization surface. Docker Compose fixes component versions and networking for reproducibility.

Table 3: Major components and use-case-specific justification

Layer	Component	Why it fits
Ingestion	Kafka 7.6.1	Back-pressure, household-key ordering, durable replay and independent consumers
Streaming	Spark 3.5.1	Event-time windows, watermarking, schema parsing and scalable aggregations
Orchestration	Airflow 2.10.2	Visible five-minute simulated-day schedule, retries and task evidence
Storage	PostgreSQL 16	Relational tariff join, constraints, indexed latest-view queries and upserts
Serving	FastAPI	Lightweight live and daily JSON APIs plus generated OpenAPI documentation
Monitoring	Prometheus/Grafana	Numeric time-series, rule evaluation and operator dashboard foundation

5 Data Ingestion and Processing

The streaming generator emits twenty readings per cycle. Values are deliberately variable; solar is zero outside daylight hours. Each record includes an ISO-8601 UTC timestamp and UUID. Kafka

keys preserve order for a household while allowing future partition parallelism. Structured JSON logs include the event and household identifiers so a demonstration can trace records.

Spark applies an explicit schema rather than inferring JSON, rejects missing identifiers and negative energy, parses event time, and calculates

$$net_grid_kwh = \max(0, consumption_kwh - solar_generation_kwh).$$

It deduplicates UUIDs and groups valid events into one-minute event-time windows by zone. Aggregates include consumption, solar, net grid draw, distinct active households and $100 \times solar/consumption$. Records later than the two-minute watermark may be excluded from finalized windows; this is a stated freshness-versus-completeness choice.

The tariff generator writes twenty rows with lifeline, standard or peak rates. Publishing the rows to Kafka preserves the source change history; loading the same file into `tariffs` creates the serving snapshot. Airflow then joins daily readings to that snapshot and upserts `daily_billing`. A rerun does not create duplicate bills because the key is report date plus household.

6 Storage and Serving Design

PostgreSQL separates atomic readings, time-window metrics, current tariffs and daily bills. A UUID primary key prevents duplicate meter events. Time/zone and latest-zone indexes match the main read patterns. Check constraints reject negative energy at the final boundary. The live endpoint uses `DISTINCT ON` with descending event time to return the newest window for every zone. The billing endpoint orders by estimated bill so high-cost households are immediately visible.

The daily charge is $net\ grid\ kWh \times tariff\ rate$. This is intentionally an estimate: export credits, taxes, standing charges and interval-specific prices are excluded. The API health endpoint executes a database query, so it detects dependency failure instead of only proving that the web process exists.

7 Observability and Failure Handling

Each Python stage emits JSON with time, severity, component and context. Kafka offsets and Spark checkpoint progress show whether consumption advances. Airflow records task state and duration for the scheduled path. FastAPI exports request counters and latency histograms. Prometheus scrapes every ten seconds.

Two rules meet and exceed the basic alert requirement. *SmartGridApiDown* becomes critical when the API target is absent for one minute. *SmartGridApiErrorRate* warns when server errors exceed 5% for two minutes. In production, further rules would detect unchanged Kafka offsets, excessive consumer lag, missing daily files, stale zone metrics, checkpoint failures and reconciliation row-count differences.

Failure recovery is layered. Kafka buffers while Spark is unavailable; Spark resumes from its checkpoint. Producer idempotence reduces duplicates during retry, and event UUIDs stop duplicate storage. Airflow tasks can retry because tariff and report writes are upserts. PostgreSQL persistence resides in a named volume. Poison events are filtered in this version; production would send them to a dead-letter topic with validation reasons and alert on its rate.

8 Results and Validation

The repository includes a representative daily report. The three sample rows demonstrate different tariffs and renewable contributions. For H003, 18.420 kWh consumption minus 4.815 kWh solar

yields 13.605 kWh net draw; at 0.31 currency units/kWh the rounded estimate is 4.22. H007 receives the lifeline rate and has a substantially smaller estimate.

Table 4: Representative consolidated daily output

Household	Zone	Use	Solar	Net grid	Tier	Bill
H003	west	18.420	4.815	13.605	peak	4.22
H007	west	12.160	5.230	6.930	lifeline	0.83
H010	east	15.780	8.620	7.160	standard	1.58

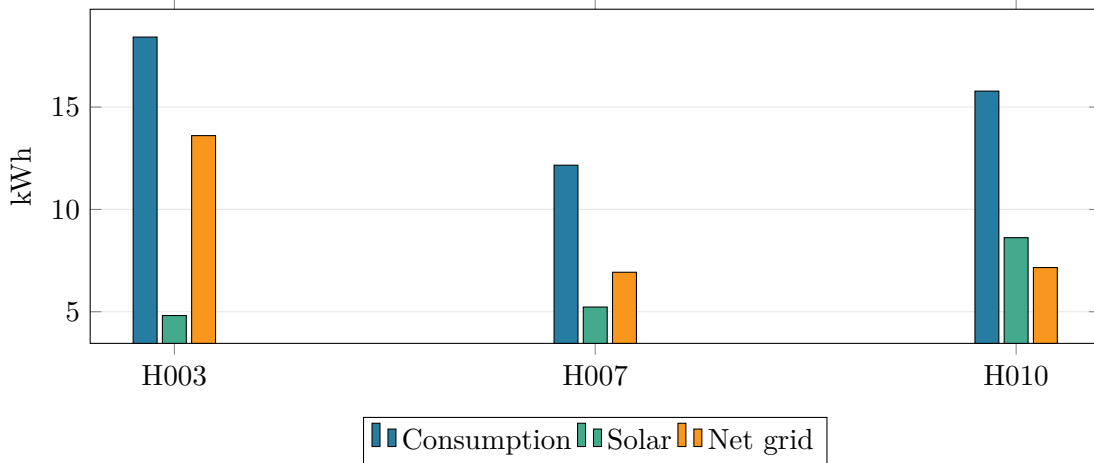


Figure 2: Sample household energy reconciliation

Automated tests validate source fields, non-negative readings, all twenty tariff rows and the zero-floor net-grid formula. The demo smoke script checks health, live output and Prometheus exposition. Operational validation should also confirm all Compose services are healthy, Spark logs show completed batches, the Airflow DAG succeeds, a CSV appears, and Prometheus shows the API target as up.

9 Security, Governance and Data Quality

The local credentials are intentionally non-secret demonstration defaults and must never be reused. Production would place credentials in a secret manager, enable TLS and SASL for Kafka, use least-privilege service accounts, restrict network paths, rotate credentials and audit API access. A schema registry would enforce compatible meter and tariff evolution. Data-quality controls would include expected household coverage, tariff range checks, timestamp drift, physical bounds, duplicate ratios and reconciliation totals. Retention and deletion policies would be attached to raw events and household reports; the design avoids personal names and addresses but household identifiers remain potentially sensitive.

10 Limitations and Production Evolution

The single Kafka broker, local Spark process, SQLite-backed Airflow metadata and one PostgreSQL node provide no high availability. Random data does not reproduce spatial or seasonal correlation. Spark's append then aggregate writes are not a single cross-table transaction; a crash between them can create a temporarily inconsistent view. The reference event stream is published but the demonstration reconciliation deliberately reads the relational current snapshot. Grafana provisions a

datasource, not a polished domain dashboard, because the assessed business outputs are the API and report.

At production scale we would deploy replicated Kafka with topic compaction for tariffs, Spark on Kubernetes with remote checkpoints, Airflow with PostgreSQL metadata and distributed workers, and managed PostgreSQL with partitioning and replicas. Raw events would also land in versioned Parquet on object storage to support inexpensive long-term replay. A schema registry, dead-letter topic, contract tests, OpenTelemetry traces and end-to-end freshness service-level objectives would strengthen governance and diagnosis. Billing would use interval tariffs, export credits and immutable tariff-effective periods rather than only the latest rate.

11 Team Contributions and Delivery Plan

The implementation is scoped for two members with balanced ownership and cross-review. Names and student IDs must replace the placeholders before submission.

Table 5: Two-person contribution statement

	Primary implementation	Report/demo responsibility
Member 1 [Name/ID]	Meter simulator, Kafka, Spark transformations, checkpointing, tests	Architecture decision, processing narrative, Spark demonstration
Member 2 [Name/ID]	Tariff source, Airflow, PostgreSQL, API, Prometheus/Grafana, Compose	Stack rationale, observability/results, scheduled-report demonstration
Joint	Integration, code review, failure testing	Final editing, limitations, rehearsal; both defend all core logic

12 Conclusion

Smart Grid Pulse satisfies the brief with two simulated source modes, Kafka ingestion, meaningful event-time processing, an Airflow-orchestrated daily feed, queryable storage, live and consolidated outputs, structured logs, metrics and alerts. Kappa is the better architectural fit because replayable events and one transformation path meet the latency and consistency needs without Lambda's duplicate logic. The solution is deliberately honest about laptop-scale compromises while preserving a clear route to a resilient production platform.

A Reproduction and Demo Checklist

Copy `.env.example` to `.env`; run `docker compose up -build -d`; wait for health checks; enable and trigger `smart_grid_daily`; inspect API docs on port 8000, Airflow on 8080, Prometheus on 9090 and Grafana on 3000. Run `python scripts/demo_check.py`. Show producer and Spark JSON logs, a successful Airflow graph, live zone results, the generated report and Prometheus rules. Use `docker compose down` after the demo. Full commands and troubleshooting assumptions are in the root README.

B Core Data Contracts

Meter events contain `event_id`, `meter_id`, `household_id`, consumption, solar generation, zone and timestamp. Tariff rows contain household, numeric rate, tier, subsidy flag and effective date. Live metrics contain window bounds, zone, consumption, solar, net draw, active households and renewable

percentage. Daily billing contains report date, household, zone, energy totals, tariff attributes and rounded estimated bill.